

GenAI Assignment

Purpose

We're looking for someone who can turn a product challenge into a working GenAI- powered solution - fast, smart, and scalable.

This assignment is designed to assess your hands-on skills with LLMs, your ability to design and implement AI-driven features, and your product-thinking mindset.

The Challenge: Build a Cybersecurity Copilot (PoC)

Imagine you're joining our team.

You've been tasked with building a small-scale prototype of an internal AI assistant for cybersecurity analysts. Its job is to support investigations by helping analysts quickly understand security incident reports and suggest next steps.

The copilot should:

1. Summarize a security incident report (written in free text)
2. Suggest a relevant mitigation plan or response strategy
3. Retrieve similar incidents from a knowledge base using vector search/RAG. (**Bonus** – implement hybrid search for both semantic and keyword-based search)

What to Deliver

Please submit a short document (PDF or Markdown/Notion link) with the following sections:

1. Solution Design & Architecture
 - Describe how your copilot works - from input to output
 - List tools, APIs, and frameworks used (e.g., OpenAI, Claude, LangChain, vector DBs)
 - Show a simple diagram or flowchart, if helpful
 - Explain your design decisions and tradeoffs (e.g., model choice, cost/performance, scalability)
2. Prompt Engineering
 - Include at least two real prompts (e.g., for summarization and mitigation)
 - Clearly explain your use of system messages, formatting, examples (few-shot?), etc.
 - Edge case handling: Show how you'd handle noisy or incomplete reports (include one such sample)
3. Sample Output



- Provide 1–2 input examples (can be made-up) and the assistant’s response
 - Mention which model/API you used to generate this output
4. Working Prototype (Required)
- Share a basic implementation of the core logic (Python)
 - At minimum, include working code or flow for:
Input → Prompt → LLM → Output (summary + mitigation)
 - You may use Jupyter notebooks, Replit, or any method that clearly demonstrates your work
5. Retrieval Pipeline Implement a basic RAG component that:
- Embeds a small dummy set of incidents (10–20)
 - Accepts a query and returns the top 1–2 relevant matches
 - You may use any tool (e.g., FAISS, Pinecone, Chroma, Weaviate, etc.)
6. Evaluation & Cost Awareness
- How would you evaluate this assistant in production?
 - Metrics for performance, hallucination risk, user value
 - Estimate rough token/cost usage per run
 - How would you monitor performance (latency, failure, drift)?

Submission Guidelines Format:

- PDF or public link (Notion, Google Doc, or GitHub)
- Duration: ~3–5 hours total

Note: It’s okay if not everything is polished or “production-ready.” We care about your approach, clarity, and hands-on ability - not perfection

